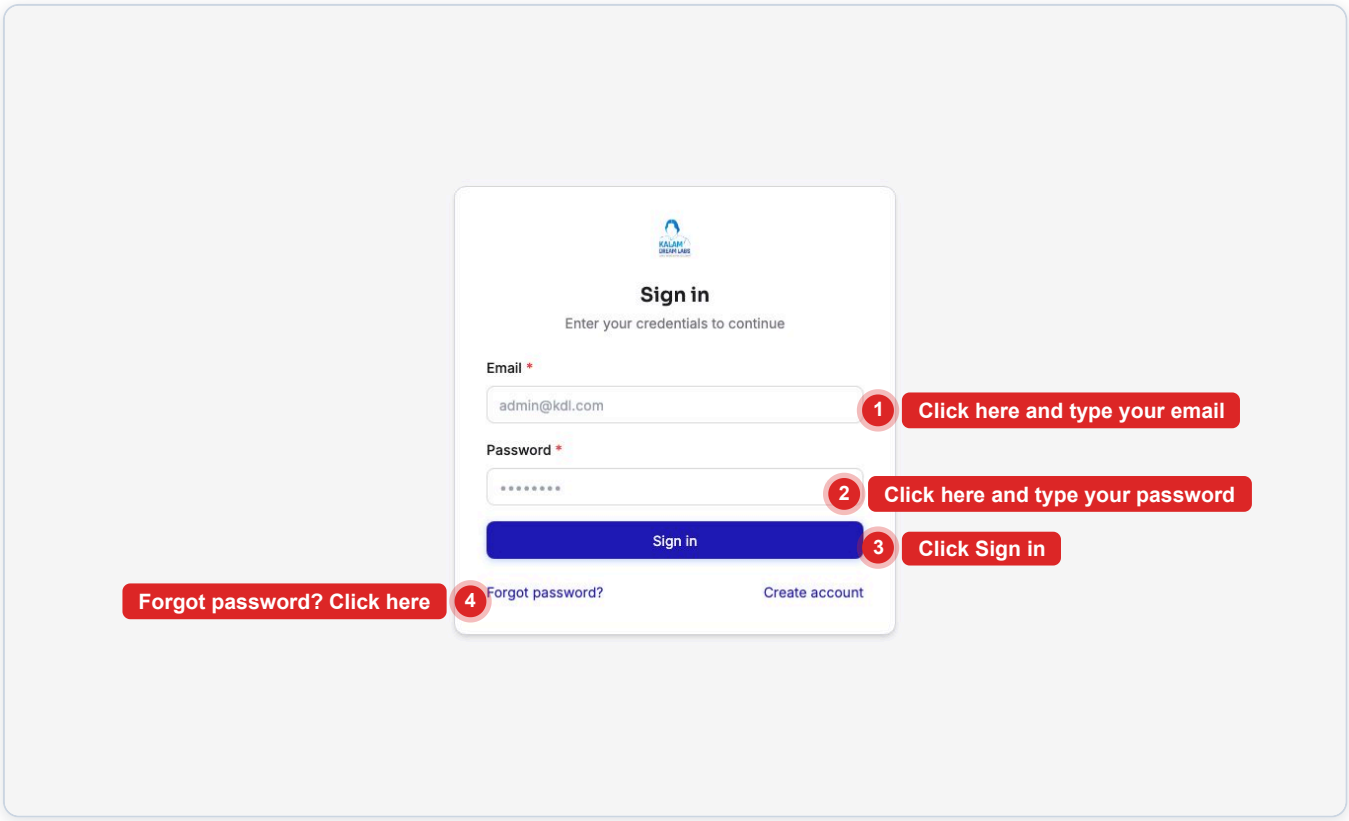
Step 1. Sign in

> KDL OS must already be installed and running. If not, follow the **Installation** guide first (Admin > Documents > Installation).

Open **http://localhost/login**. Email `admin@kdl.com`, password `kdl-dev-seed-password`. KDL OS asks you to choose a new password on first sign-in.



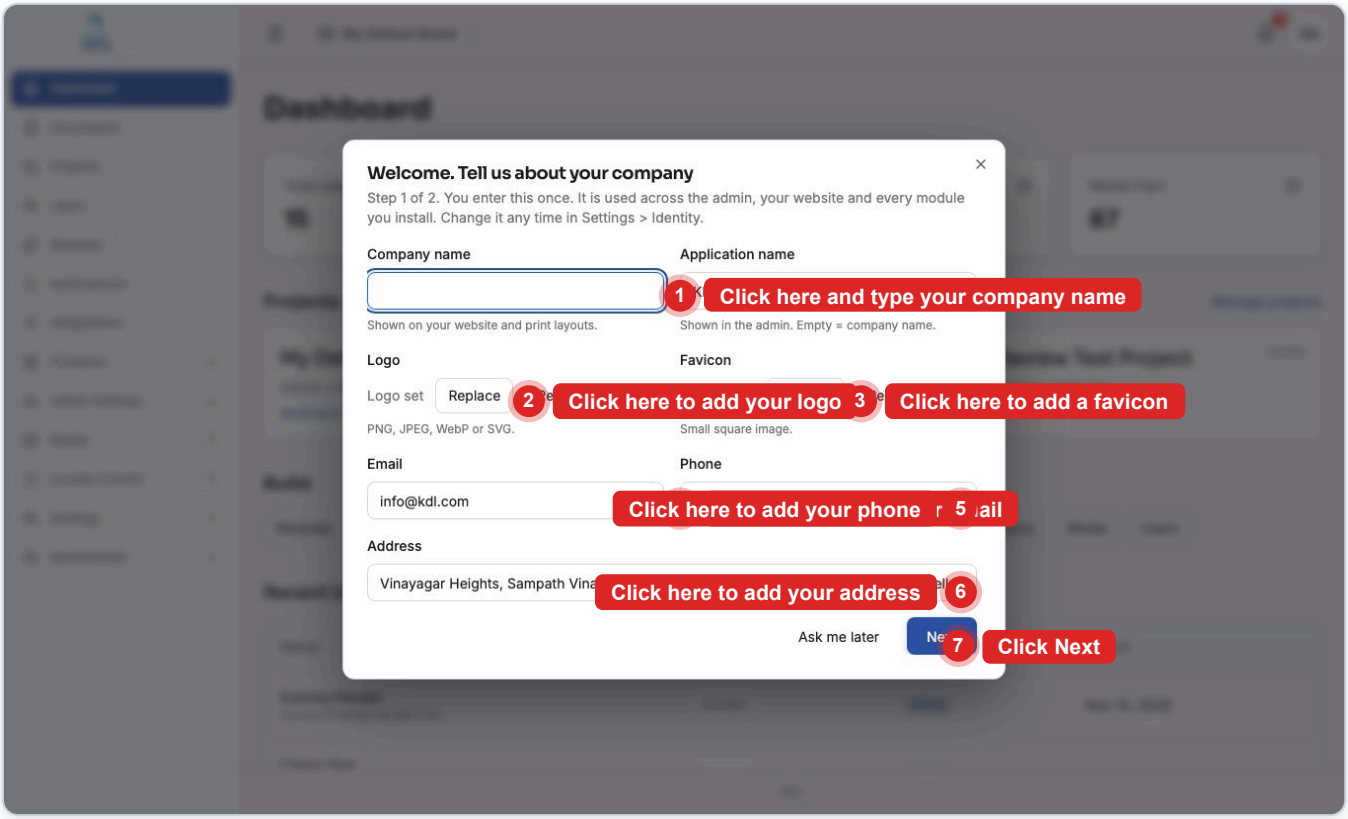
The sign-in screen

> Forgot the password or locked out? `docker compose exec backend node scripts/reset-admin-password.js --email admin@kdl.com --password 'Admin@123'` (at least 8 characters), then sign in with it. 429 or "Account locked" while testing: wait a minute or `docker compose restart backend`.

Step 2. Fill in the setup wizard

On a fresh install a **Setup wizard** opens over the dashboard. Step 1 asks for your company: click each field and type the **Company name**, application name, email, phone and address, and click **Replace** to add a logo and favicon. Click **Next**.

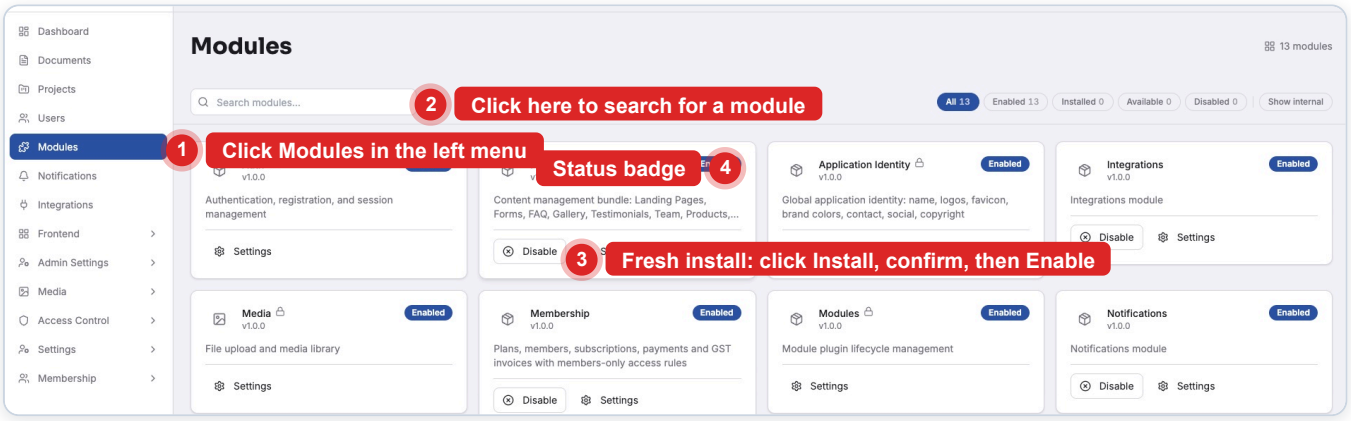
Step 2 asks what you are building. Click the **Website** card to tick it (leave Landing Pages unticked), then click **Next**. If the next window offers a starter website, click **Yes, create my website**: KDL OS builds a ready 5-page site (Home, About, Services, Contact, FAQ) that you can edit.



Setup wizard, step 1

Step 3. Open Modules, install and enable Frontend

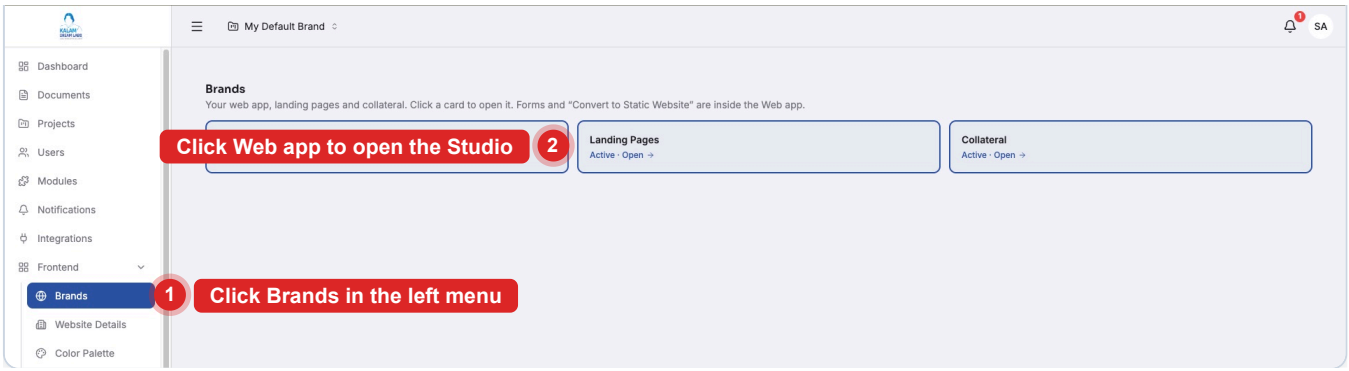
Click **Modules** in the left menu. Find the **Frontend** card (use the search box if you need to). On a fresh install the card shows **Install**: click it and confirm, then click **Enable** and confirm. The badge changes to **Enabled**.



Modules: the Frontend card

Step 4. Open the Studio

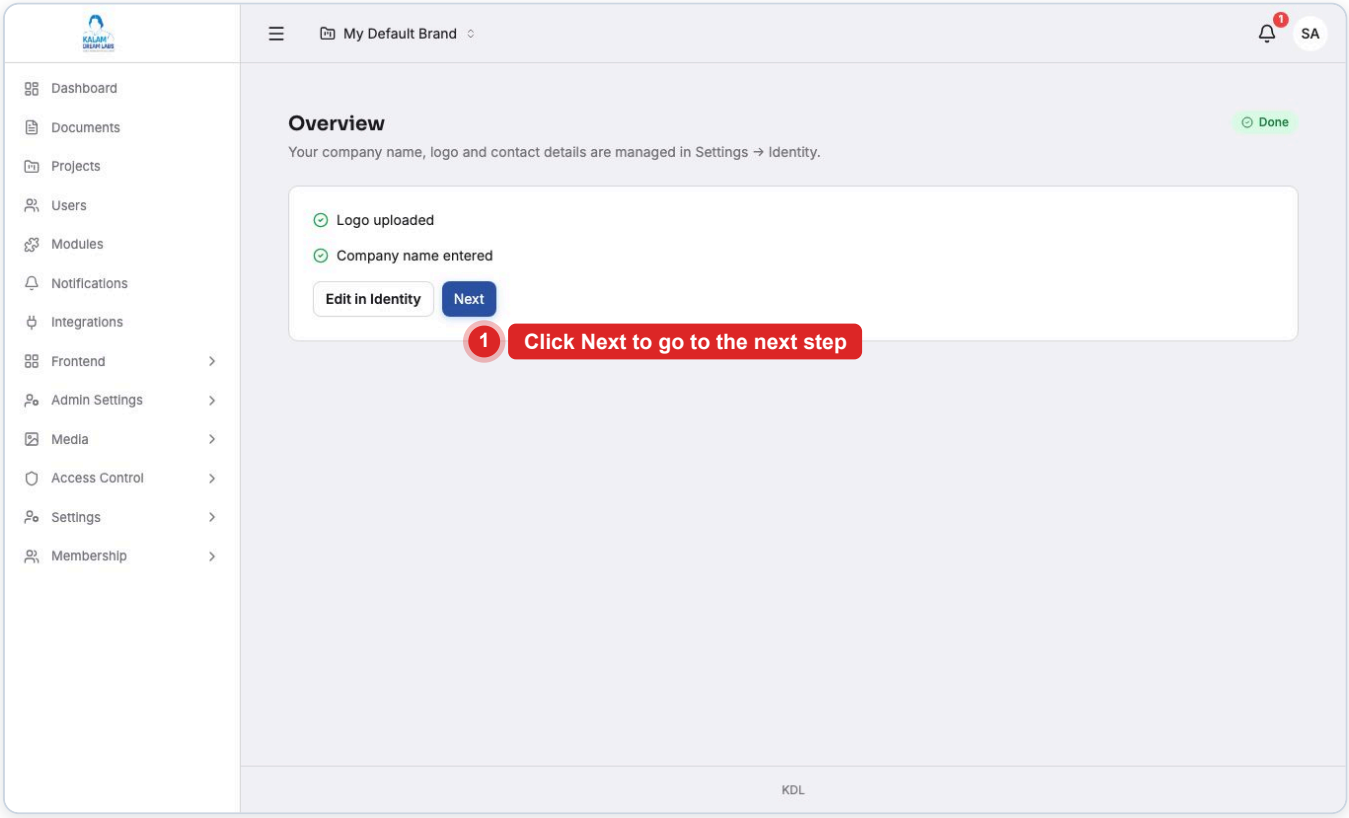
Click **Frontend > Brands** in the left menu, then click the **Web app** card. This opens the Studio for your website. The first time, click **Start Studio run**. Tabs along the top: Overview, Color Palette and Website.



Brands: open the Web app

Step 5. Studio Overview: logo and company details

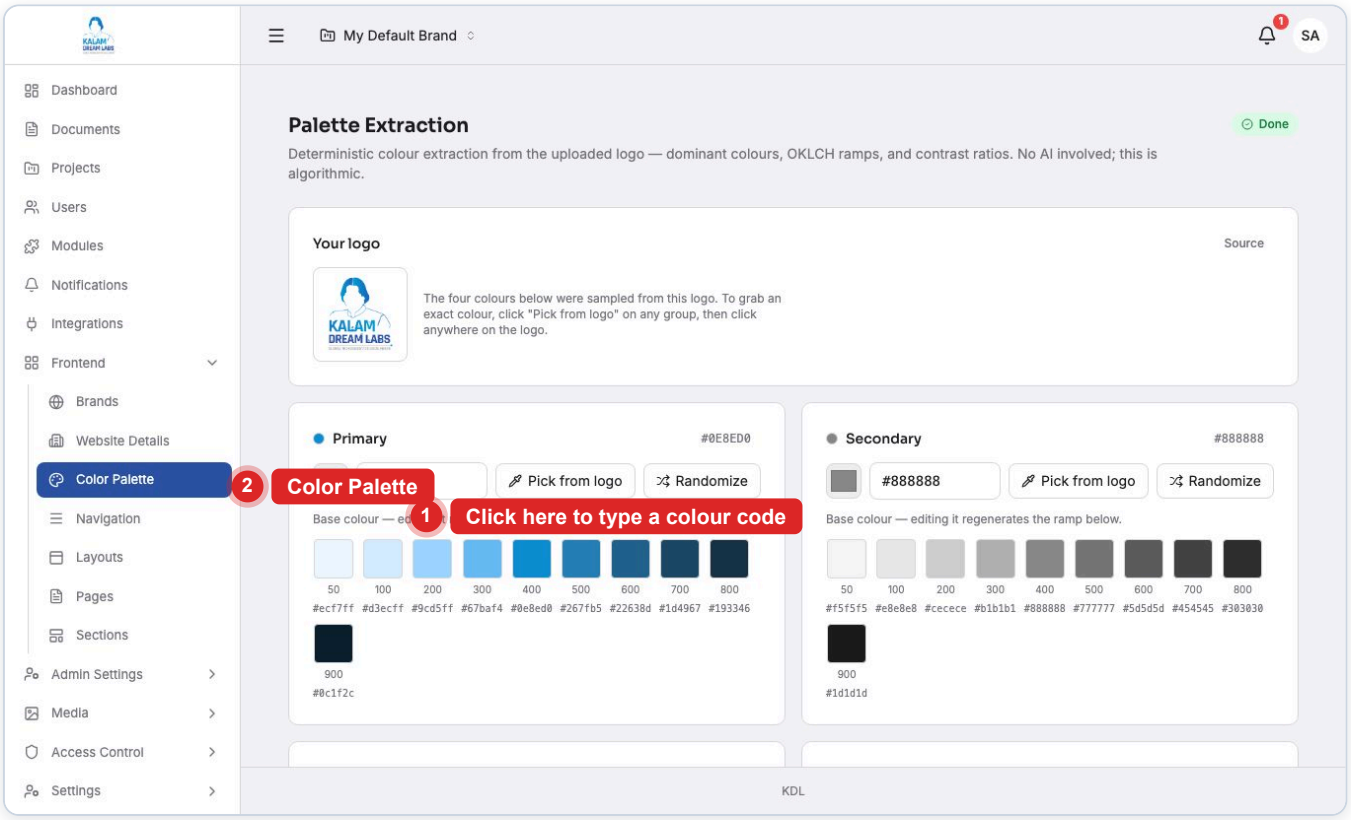
Upload your logo (svg, png, jpeg or webp), check the company and contact details, then click **Next**. Use **Edit in Identity** to change the details. The logo also becomes the admin logo.



Studio: Overview

Step 6. Color Palette

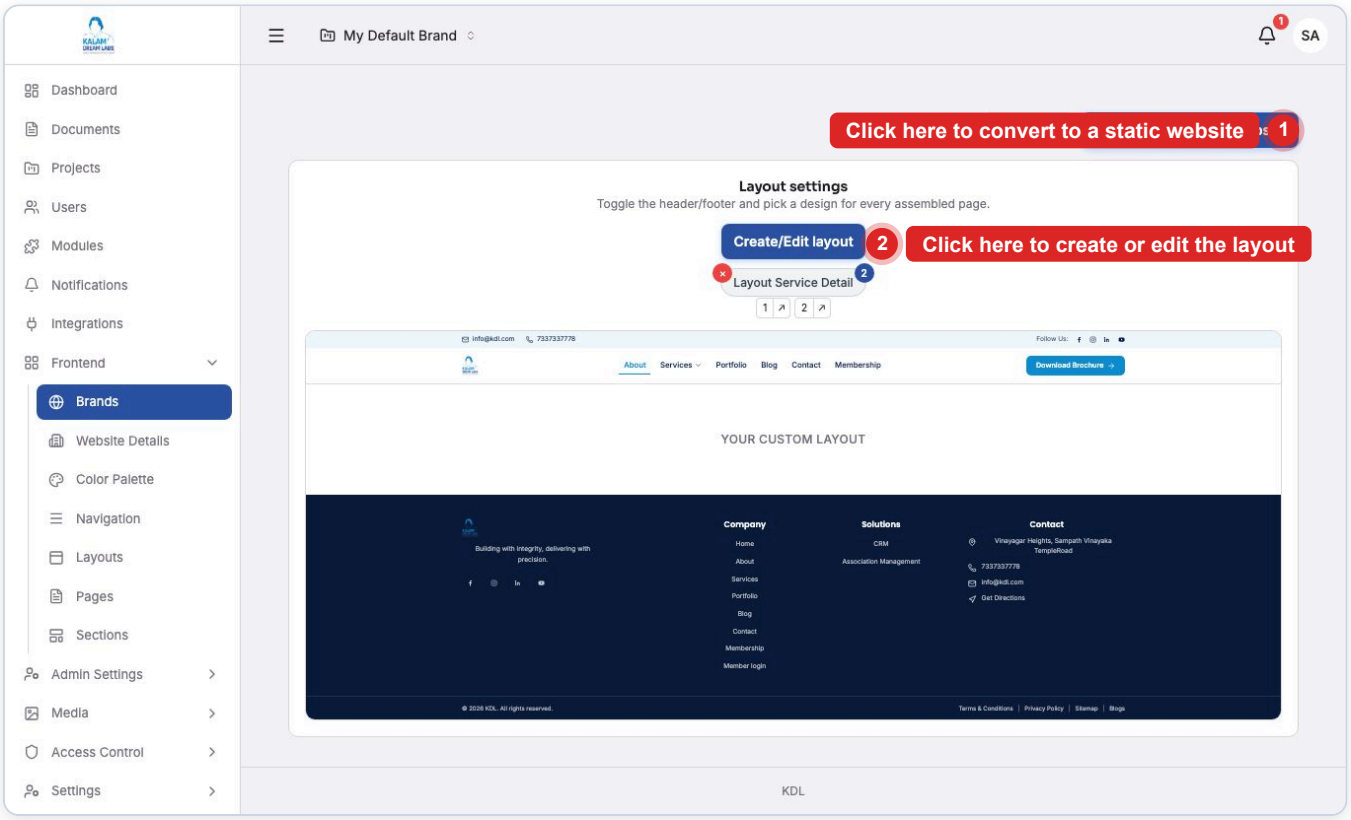
Four brand colours are taken from your logo: Primary, Secondary, Tertiary and Quaternary. Keep them, or click a colour box and type a new code, click **Pick from logo** or **Randomize**. Then click **Submit palette**. The colours become the website's design tokens.



Studio: Color Palette

Step 7. Website stage: layout

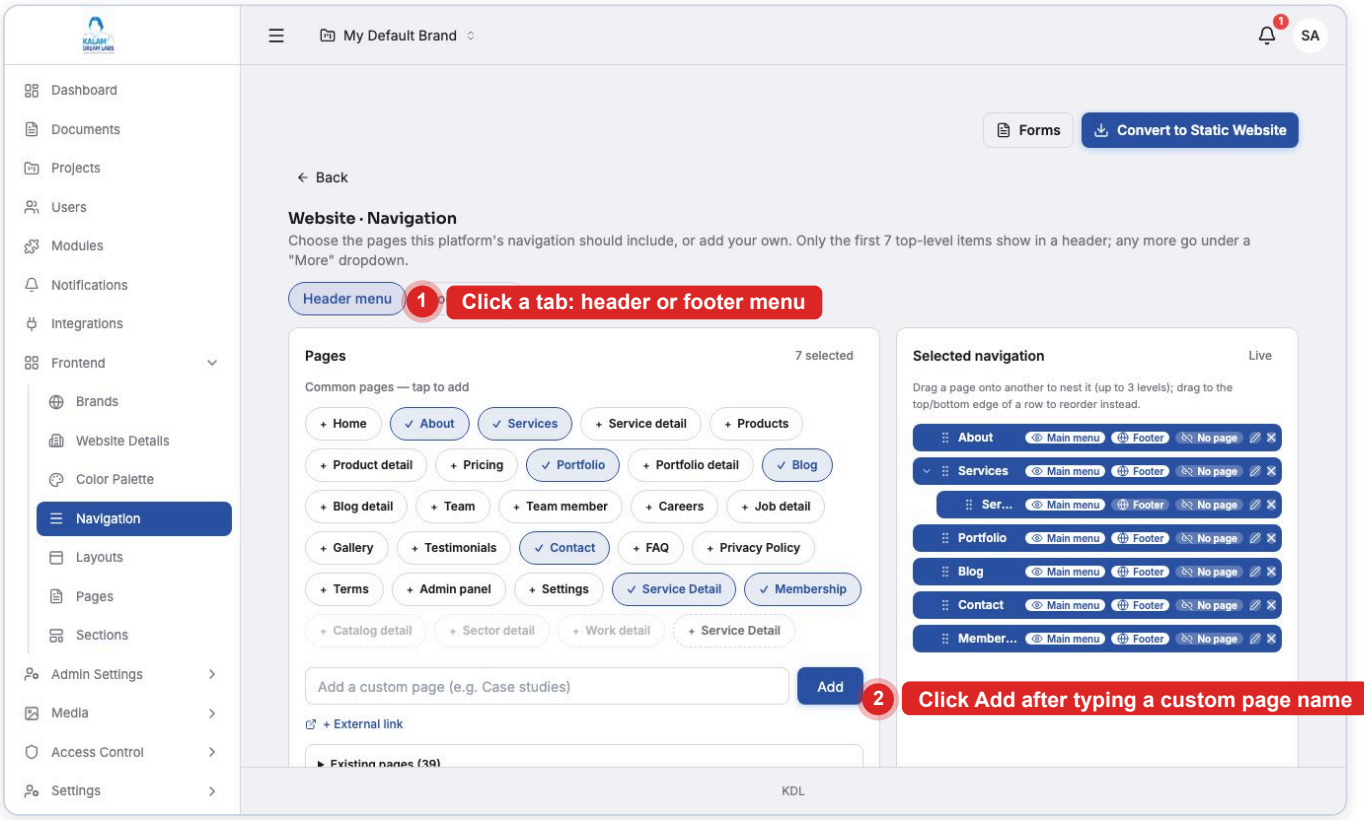
Set typography and fonts, then use **Navigation** and **Pages** to build the pages. Click **Create/Edit layout** to choose the Top header, Header and Footer. The header and footer of **Home** are shared by every public page.



Studio: Website stage

Step 8. Choose the menu pages

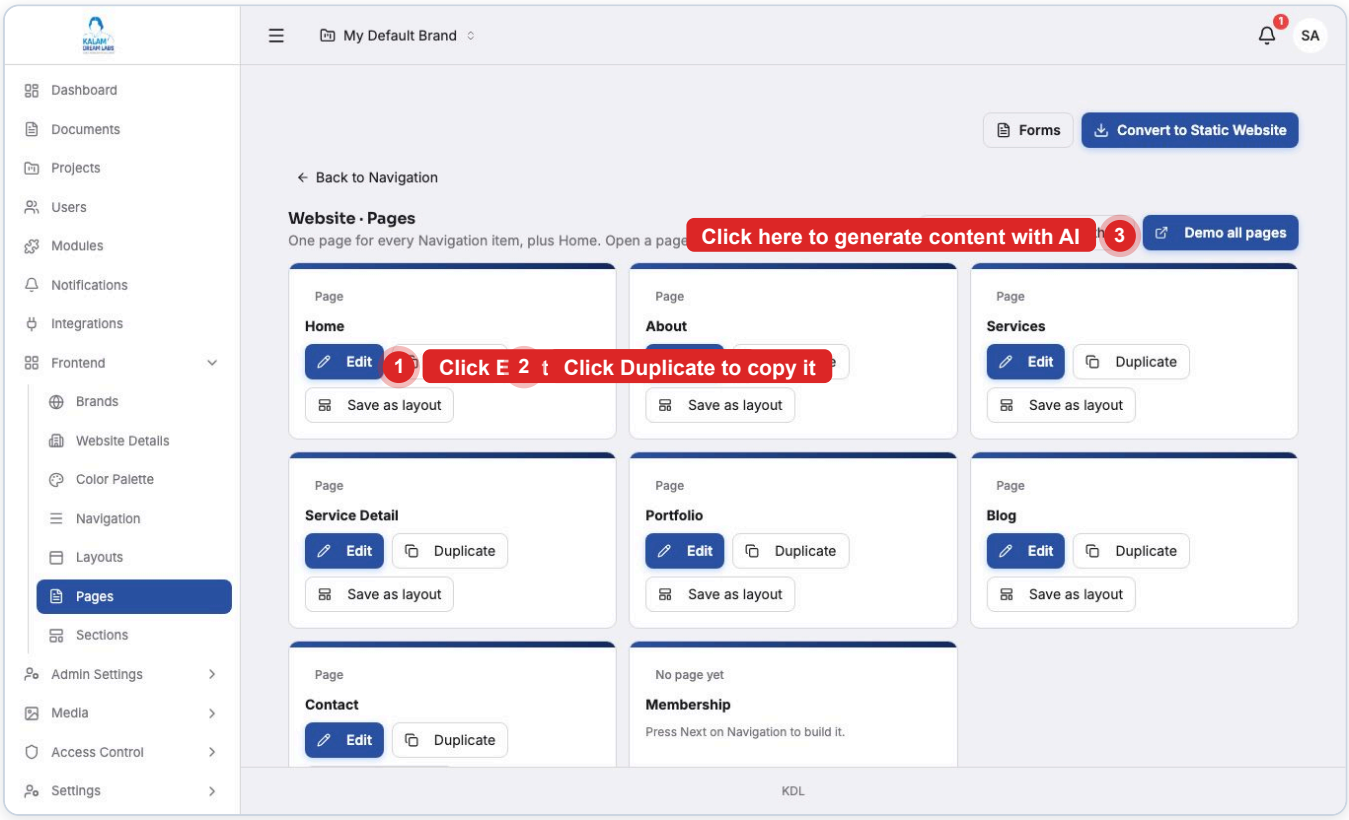
Open **Frontend > Navigation**. Click the page names you want in the menu (About, Services, Portfolio, Blog, Contact and so on), or type a custom page and click **Add**. Use the **Header menu** and **Footer menu** tabs. One page is created for every menu item, plus Home.



Frontend > Navigation

Step 9. Edit and publish the pages

Open **Frontend > Pages**. Click **Edit** on a page to change its text, images and links in the section builder, then **Save** and **Publish**. Publish stops real problems (an empty page, a missing title, a broken link) and only suggests the rest. Click **Duplicate** to copy a page. **Frontend > Sections** holds reusable sections.

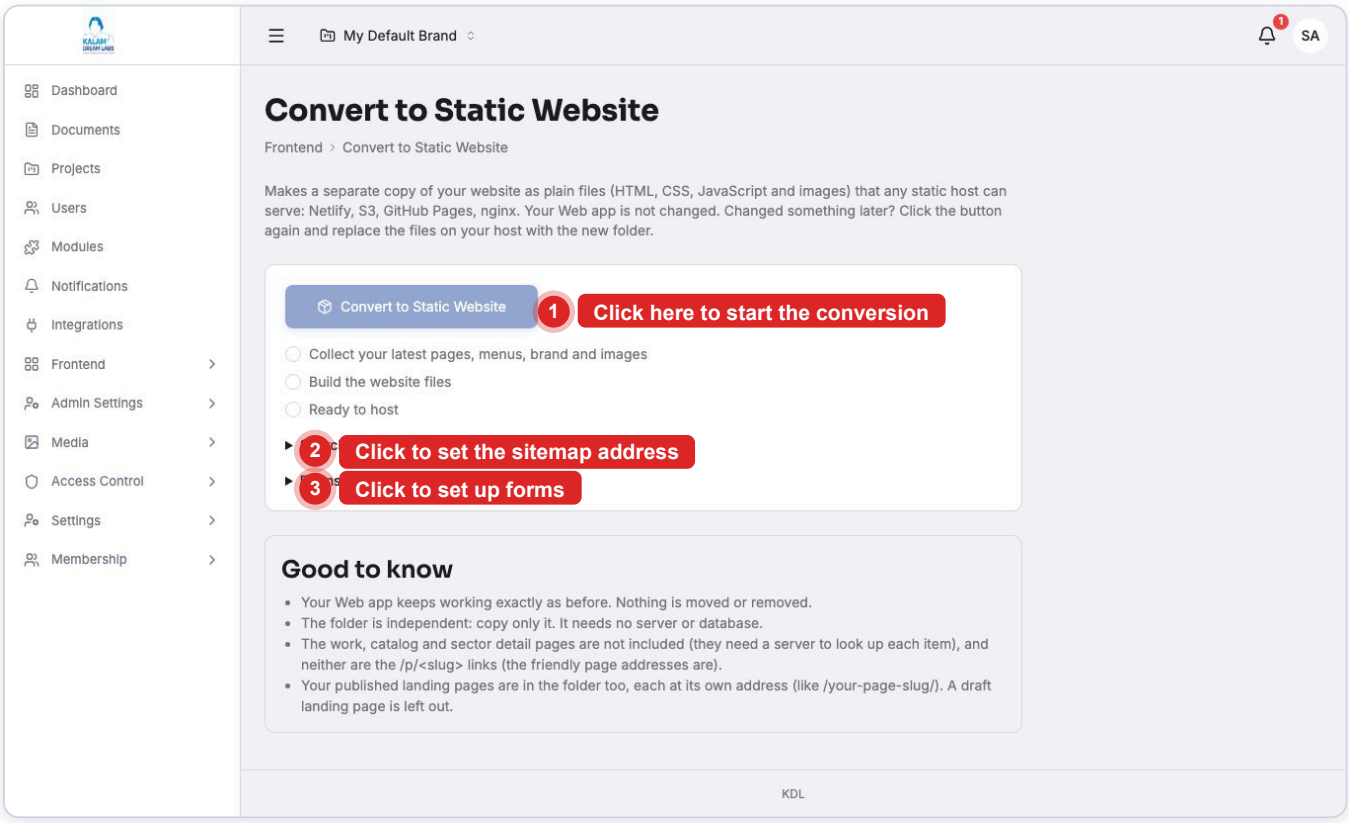


Frontend > Pages

Step 10. Convert to a static website

Static files (HTML, CSS and images) can be hosted anywhere with no server. Publish your pages first, then click **Convert to Static Website** (the button at the top of the Studio, or http://localhost:3001/admin/static-website).

Optional: type your **Website address** (for example https://www.example.com) to add a sitemap. Type your **Web app address** (https only) if forms must keep working; if you leave it empty, forms show your contact email and phone. Click **Convert to Static Website** and wait a few minutes until the card says it is ready.



Frontend > Convert to Static Website

Step 11. Download the files and host them on GitHub

Click **Download as .zip** on the same page and unzip it. The files inside, with `index.html` at the top, are your finished website.

1. On GitHub create a new empty repository, for example `my-website`.
2. Put the files in a new folder, open a terminal in it, and run (use your own username and repository name):

```
TERMINAL
git init
git add .
git commit -m "Website"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/my-website.git
git push -u origin main
```

3. On GitHub open **Settings > Pages**, choose **Deploy from a branch**, pick **main** and the **/ (root)** folder, then **Save**.
4. After a minute your website is live at `https://YOUR-USERNAME.github.io/my-website/`.

Notes: Changing things later

This is not a step. Use it when something changes.

- **Text, images, links:** Frontend > Pages, Edit, Save, Publish. No re-run is needed.
- **Menus:** Frontend > Navigation.
- **Header, top bar or footer:** Studio > Layouts > Create/Edit layout.
- **Colours or fonts:** Studio > Color Palette, Submit palette.
- After any change: publish, then click **Convert to Static Website** again, replace the files in your GitHub folder, then `git add .`, `git commit -m "Update"` and `git push`. The static copy never updates itself.